

GAMMA · SECURITY REVIEW

Gamma

PerpetualVault mitigation review.

A security review of PerpetualVault mitigation review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Gamma	PerpetualVault mitigation review	Guardian	May 3 – May 9, 2025	May 16, 2025

§ 00

Contents.

34 pages — overview, confidence ranking, scope, 24 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 24 findings	P. 09
§ 05	Disclaimer	P. 33
§ 06	About Guardian	P. 34

§ 01

Overview.

Guardian reviewed PerpetualVault mitigation review across a single review round. The final report includes 7 Medium, 17 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	GammaStrategies/PerpetualVault
CLIENT	METHODOLOGY
Gamma	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
PerpetualVault mitigation review	ba7b4bec49c4
PREPARED BY	
Guardian	
AUDIT WINDOW	
May 3 – May 9, 2025	
FINAL REPORT	
May 16, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	7	0	7	0	0
Low	17	0	17	0	0
Informational	0	0	0	0	0
All severities	24	0	24	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed PerpetualVault mitigation review in `GammaStrategies/PerpetualVault`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Incorrect Fee Amount May Be Calculated	Logical Error	MEDIUM	ACKNOWLEDGED	p. 9
M-02	Lacking setGovernanceFee Implementation	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
M-03	Position Fees May Be Paid Twice	Logical Error	MEDIUM	ACKNOWLEDGED	p. 11
M-04	gmxClock Errantly Set To False	Warning	MEDIUM	ACKNOWLEDGED	p. 12
M-05	Excessive Refund During Liquidations	Logical Error	MEDIUM	ACKNOWLEDGED	p. 13
M-06	Liquidations During Withdrawal Settlement Loop	Warning	MEDIUM	ACKNOWLEDGED	p. 14
M-07	totalShares Accounting Perturbed By CancelFlow	Logical Error	MEDIUM	ACKNOWLEDGED	p. 15
L-01	Outdated Reader Used	Warning	LOW	ACKNOWLEDGED	p. 16
L-02	Outdated NatSpec	Documentation	LOW	ACKNOWLEDGED	p. 17
L-03	_swapOutIndexToken DoS	DoS	LOW	ACKNOWLEDGED	p. 18
L-04	Zero Position Key Used For totalAmount	Warning	LOW	ACKNOWLEDGED	p. 19
L-05	Misleading Comment	Documentation	LOW	ACKNOWLEDGED	p. 20
L-06	Misleading Variable Name	Naming	LOW	ACKNOWLEDGED	p. 21
L-07	Fee Discounts Are Not Accounted For	Warning	LOW	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Lacking Withdrawal Execution Fee Refunds	Warning	LOW	ACKNOWLEDGED	p. 23
L-09	Unused COMPOUND Action	Superfluous Code	LOW	ACKNOWLEDGED	p. 24
L-10	Signal Change Flow Left Incomplete	Warning	LOW	ACKNOWLEDGED	p. 25
L-11	Zero Position Key Used For flowData	Warning	LOW	ACKNOWLEDGED	p. 26
L-12	Incorrect Price Impact Rounding	Rounding	LOW	ACKNOWLEDGED	p. 27
L-13	Arbitrary Price Impact Conversion	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 28
L-14	Execution Fee Charged with No GMX Call	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 29
L-15	Deposit Reverts After Full Liquidation	DoS	LOW	ACKNOWLEDGED	p. 30
L-16	ADLs May Cause Excessive Fees	Warning	LOW	ACKNOWLEDGED	p. 31
L-17	totalDepositAmount Not Decrementd In Insolvency	Logical Error	LOW	ACKNOWLEDGED	p. 32

MEDIUM **M-01**

ACKNOWLEDGED

Incorrect Fee Amount May Be Calculated

CATEGORY	Logical Error
LOCATION	https://github.com/GammaStrategies/PerpetualVault/blob/ba7b4bec49c42ed1ba440f5d00e8169ca39dabbb/contracts/PerpetualVault.sol#L486
REVIEW	Main Review

— DESCRIPTION

GMX is moving to a `balanceWasImproved` model instead of a `hasPositiveImpact` approach for the position fees. This upcoming change is listed here: <https://github.com/gmx-io/gmx-synthetics/pull/93>

However the Gamma contracts assume that the `hasPositiveImpact` approach is still being used. As a result, for markets with no price impact configured the fee calculated by Gamma will not match the fee calculated by GMX.

Additionally, when an order flips the skew, moving the absolute skew closer to balanced the order may improve the skew but still receive negative impact due to the negative impact factor being larger than the positive impact factor. Orders which meet this criteria will have the fee overestimated for the user.

— RECOMMENDATION

The `balanceWasImproved` value for the order is not easily accessible without comparing the pool state directly before and after an order's execution. Therefore the best course of action may be to either ensure that no zero slippage GM markets are used with the Gamma system, or to acknowledge the imperfection in the fee calculations for these markets.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Lacking setGovernanceFee Implementation

CATEGORY Logical Error
LOCATION PerpetualVault.sol: 710
REVIEW Main Review

DESCRIPTION

The `setGovernanceFee` function lacks an implementation to assign the related `governanceFee` value.

RECOMMENDATION

Implement the logic to assign the `governanceFee` value in the `setGovernanceFee` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Position Fees May Be Paid Twice

CATEGORY	Logical Error
LOCATION	PerpetualVault.sol: 1120
REVIEW	Main Review

— DESCRIPTION

In the `_withdraw` function the `feeAmount` is removed from the `collateralDeltaAmount` regardless of the sign and magnitude of the pnl.

If the pnl is positive and the `swapPnlToCollateral` token swap succeeds, the profit will be used to cover the fee amount in the decrease order execution. As a result the withdrawer would pay the position fee by having it deducted from the `collateralDeltaAmount` specified by Gamma and also by having it deducted by the pnl amount returned by GMX.

— RECOMMENDATION

If the position pnl is positive then account for the fact that the positive pnl amount can be used to cover the feeAmount.

However be aware that the positive pnl amount cannot be used to cover the feeAmount if the current position is long and the `swapPnlToCollateral` swap fails. If this swap is assumed to always succeed in the system, then consider carefully monitoring for the failure of this swap, denoted by the `SwapReverted` event emitted by GMX, and taking manual action.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

gmxLock Errantly Set To False

CATEGORY Warning
LOCATION PerpetualVault.sol
REVIEW Main Review

— DESCRIPTION

In the `afterLiquidationExecution` function the `gmxLock` variable is assigned to `false`, however it is possible that an order is still pending in GMX for the vault.

This may lead to unintended consequences such as un-cancellable orders through the `cancelOrder` keeper function or accidental order creation by the keeper when one is already pending in GMX.

— RECOMMENDATION

Remove the `gmxLock` assignment in the `afterLiquidationExecution`.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Excessive Refund During Liquidations

CATEGORY Logical Error
LOCATION PerpetualVault.sol
REVIEW Main Review

DESCRIPTION

In the `_withdraw` function, if a liquidation occurred and the `curPositionKey` has been assigned to zero then `refundFee` is assigned as true in the `_handleReturn` invocation.

As a result, a refund may be granted to the user when the settle action in GMX has already taken place for their withdrawal.

Consider the following scenario:

- User A initiates a withdrawal
- The settle decrease order is submitted and is executed in GMX
- The `nextAction.selector` is a withdrawal action
- Before the Gamma keeper executes the next action, the vault position is liquidated and the `curPositionKey` is set to 0.
- The Gamma keeper executes the next action and the `_withdraw` invocation invokes a `_handleReturn` with `refundFee` as true.
- The user is refunded for the majority of their executionFee, such that refund + executionFee paid to GMX for the settle action is greater than the executionFee initially paid by the user.

RECOMMENDATION

When the `_withdraw` function is invoked in `runNextAction` and the vault position has been liquidated consider not refunding the fee in this case, since the settle action will have already taken place.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Liquidations During Withdrawal Settlement Loop

CATEGORY Warning
LOCATION PerpetualVault.sol
REVIEW Main Review

DESCRIPTION

When a liquidation occurs during the withdrawal flow, it is possible for the settlement action to get stuck and continue to fail and be retried in a cycle.

Consider the following order of events:

- A user initiates a withdrawal, the settlement action is created in GMX.
- The vault position is liquidated in GMX, the `afterLiquidationExecution` sets the `nextAction` to be `NextActionSelector.WITHDRAW_ACTION` to restart the withdrawal and allow the user to exit.
- The settlement action is executed after the liquidation and is therefore cancelled.
- In the `afterOrderCancellation` callback the `nextAction` is reset to `NextActionSelector.SETTLE_ACTION`, therefore overwriting the `nextAction` that was set by the liquidation callback.
- The settlement order is submitted again, and will again fail and continue the cycle, preventing the user from withdrawing.

RECOMMENDATION

Consider adding a check in the `afterOrderCancellation` callback such that if the order was a settlement order and the `curPositionKey` has been set to zero then the next action is set to `NextActionSelector.WITHDRAW_ACTION` so that the withdrawal flow can be reset and the user can receive their funds.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

totalShares Accounting Perturbed By CancelFlow

CATEGORY Logical Error
LOCATION PerpetualVault.sol
REVIEW Main Review

— DESCRIPTION

In the `_cancelFlow` function during the final step of a deposit flow the deposit object can have a nonzero shares amount associated which is not removed from the total upon cancellation.

As a result, if a deposit is cancelled at the last step before the `FINALIZE` action, the `totalShares` accounting will be perturbed.

— RECOMMENDATION

Consider explicitly disallowing the `_cancelFlow` function from being called for deposits which have already been minted a nonzero amount of shares.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Outdated Reader Used

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The tests and PositionInfo interface relies on an outdated Reader contract that was deployed almost a year ago. This Reader may become deprecated with future releases and increases risk of an interface mismatch bug.

— RECOMMENDATION

Consider updating to the most recent Reader deployment seen here: <https://github.com/gmx-io/gmx-synthetics/blob/updates/deployments/arbitrum/Reader.json>

And ensure that the interfaces in that Reader deployment match the interfaces used by Gamma, namely the inclusion of a positionKey bytes32 variable at the start of the PositionInfo struct:

<https://github.com/gmx-io/gmx-synthetics/blob/cb47fb783b017cfa815a353f6f39d415f60646e8/contracts/reader/ReaderPositionUtils.sol#L20>

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Outdated NatSpec

CATEGORY	Documentation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The NatSpec across many functions in the Gamma codebase is outdated.

— RECOMMENDATION

Consider updating the NatSpec documentation for each function in the Gamma PerpetualVault codebase.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

_swapOutIndexToken DoS

CATEGORY	DoS
LOCATION	PerpetualVault.sol: 358
REVIEW	Main Review

— DESCRIPTION

In the `runNextAction` function the `_swapOutIndexToken` function is used to swap any remaining index tokens from the contract balance to the collateral token.

When submitting the `runNextAction` transaction to the mempool, the keeper may not supply any `metadata[1]` entry if it doesn't observe any index token contract balance.

However a malicious actor could frontrun the `runNextAction` transaction and supply index tokens to the contract in order to cause the `_swapOutIndexToken` logic to occur and therefore revert when the `metadata[1]` entry is decoded, thus censoring the `runNextAction` invocation.

— RECOMMENDATION

Be aware of this DoS vector, on Arbitrum this is not currently a concern, however for any Gamma vaults on Avalanche this is noteworthy. Consider ensuring the keeper always provides a `metadata[1]` entry to handle even malicious `indexToken` transfers.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Zero Position Key Used For totalAmount

CATEGORY	Warning
LOCATION	PerpetualVault.sol: 826
REVIEW	Main Review

— DESCRIPTION

In the `_totalAmount` function when the `positionIsClosed` value is `false` the `positionData` is always queried from GMX for the `curPositionKey`.

However there are cases such as liquidation where the `curPositionKey` is reset to 0, while `positionIsClosed` is intentionally not marked as `true`.

— RECOMMENDATION

No cases have been identified where the `curPositionKey` is zero and `positionIsClosed` is false for a leveraged position during an invocation of the `_totalAmount` function. However out of an abundance of caution, consider handling the zero `curPositionKey` case to avoid any unintended consequences of future updates.

When the `curPositionKey` is 0 the `_totalAmount` function should not consult GMX for the position value with the `getPositionInfo` function.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Misleading Comment

CATEGORY Documentation
LOCATION GmxProxy.sol: 530
REVIEW Main Review

DESCRIPTION

The documentation for the `fundEth` function mentions that the `GmxProxy` attempts to avoid native ether from GMX. However the `GmxProxy` specifically wants to receive native Ether from GMX and does so by implementing the `refundExecutionFee` function.

RECOMMENDATION

Remove the misleading `fundEth` comment.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Misleading Variable Name

CATEGORY Naming
LOCATION VaultReader.sol: 52
REVIEW Main Review

DESCRIPTION

In the `getPositionInfo`, `getNegativeFundingFeeAmount`, and `getPnl` functions the `sizeInTokens` variable is assigned as the result of the `getPositionSizeInUsd` function.

RECOMMENDATION

Correct the `sizeInTokens` variables to be the result of the `getPositionSizeInTokens` function.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Fee Discounts Are Not Accounted For

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

When computing the position fee during a withdrawal there is no consideration for discounts that may be granted to the vault position due to pro tiers or in the case that a nonzero referral code is used.

As a result withdrawers may be charged a slightly higher fee than what is actually levied against the vault position.

— RECOMMENDATION

Be aware of this behavior in the event that the vault receives a pro tier discount or if a referral code is used. This inaccuracy goes in the favor of the vault so it is acceptable.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Lacking Withdrawal Execution Fee Refunds

CATEGORY Warning
LOCATION PerpetualVault.sol
REVIEW Main Review

DESCRIPTION

In the withdrawal flow for perpetual positions the `refundFee` value in the `_nextAction.data` is always assigned to `false`. As a result users will never receive refunds for the overestimated portion of the execution fee they paid up front.

RECOMMENDATION

Consider if this is the expected behavior, if it is not consider assigning the `refundFee` value to `true`.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Unused COMPOUND Action

CATEGORY Superfluous Code
LOCATION PerpetualVault.sol
REVIEW Main Review

— DESCRIPTION

The `COMPOUND_ACTION` `NextActionSelector` is unused in the codebase.

— RECOMMENDATION

Consider removing the `COMPOUND_ACTION` from the `NextActionSelector` enum.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Signal Change Flow Left Incomplete

CATEGORY Warning
LOCATION PerpetualVault.sol
REVIEW Main Review

DESCRIPTION

During the signal change flow it is possible for a switch from 1x long to 1x short to be left incomplete if a GMX swap is used to close the existing position and the GMX swap is cancelled.

When the GMX Market Swap action is cancelled the `nextAction.selector` is assigned as a swap action, overwriting the increase action. Therefore when the MarketSwap is retried it will be the final action in the flow and the signal change will complete without opening the new short position.

RECOMMENDATION

Be aware of this edge case when the MarketSwap action fails during a signal change. And consider only using dex swaps to avoid this failure when changing from long 1x to short 1x.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Zero Position Key Used For flowData

CATEGORY Warning
LOCATION PerpetualVault.sol
REVIEW Main Review

— DESCRIPTION

When a liquidation occurs directly after a user initiates a deposit the `flowData` is assigned as the result of the `getPositionSizeInTokens` for a `curPositionKey` of 0.

It is unlikely that a position may exist in GMX with a `positionKey` of 0, however in the event that such a position does exist the `flowData` would be errantly assigned to a value that is not accurate to the Gamma vault position.

— RECOMMENDATION

Out of an abundance of caution in the `getPositionSizeInTokens` `VaultReader` function, return 0 if the `positionKey` provided is zero. Additionally, consider adopting the same behavior for other view functions which accept a position key as a parameter.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

Incorrect Price Impact Rounding

CATEGORY Rounding
LOCATION —
REVIEW Main Review

DESCRIPTION

The rounding used in the `getPriceImpactInCollateral` function does not match the rounding performed by GMX when determining the `sizeDeltaInTokens` for a position. As a result the `getPriceImpactInCollateral` function will slightly miscalculate the price impact result.

The `expectedSizeInTokensDelta` is calculated as:

```
uint256 expectedSizeInTokensDelta = isLong ?  
sizeDeltaInUsd / prices.indexTokenPrice.max :  
sizeDeltaInUsd / prices.indexTokenPrice.min;
```

Meanwhile GMX calculates the `sizeInTokensDelta` as:

```
if (params.position.isLong()) {  
    // round the number of tokens for long positions down  
    cache.baseSizeDeltaInTokens = params.order.sizeDeltaUsd() / indexTokenPrice.max;  
} else {  
    // round the number of tokens for short positions up  
    cache.baseSizeDeltaInTokens = Calc.roundUpDivision(params.order.sizeDeltaUsd(),  
indexTokenPrice.min);  
}
```

RECOMMENDATION

Use round up division for shorts to match GMX's calculation.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Arbitrary Price Impact Conversion

CATEGORY Unexpected Behavior
LOCATION VaultReader.sol: 171
REVIEW Main Review

DESCRIPTION

The `priceImpactInCollateralTokens` value is computed using the max or min `indexTokenPrice` and `shortToken` price depending on if the position `isLong`.

This may seem correlated to the way that the `expectedSizeInTokensDelta` is computed, however it is not. The `expectedSizeInTokensDelta` max and min price usage is simply necessary to compute the actual amount of price impact which was experienced in the GMX system.

Therefore the translation to a collateral token value of the price impact index token amount is arbitrarily assigned by the Gamma system. The calculation currently decides to use a maximum and minimum price interchangeably based upon `isLong`.

However, technically the most resilient method would be to use the price combination that rounds the amount of price impact experienced towards negative infinity. This follows the best practice of rounding in favor of the protocol over the favor of the user.

RECOMMENDATION

Consider implementing rounding in the favor of the protocol by assigning the minimum value of both calculations as the resulting `priceImpactInCollateralTokens`.

RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Execution Fee Charged with No GMX Call

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/PerpetualVaultperpetual-vault-rem-team1/blob/ba7b4bec49c42ed1ba440f5d00e8169ca39dabbb/contracts/PerpetualVault.sol#L1110
REVIEW	Main Review

— DESCRIPTION

When a liquidation reduces the GMX position to zero, the `curPositionKey` is deleted and deposits are paused. However, since `positionIsClosed` remains false, users who withdraw are still charged an execution fee under the assumption that GMX calls like `settle` and `withdrawGMX` will be executed.

Since the `curPositionKey` is empty, these GMX calls are skipped. The `_handleReturn` function does issue a refund, but because no callback occurred, the refunded amount is incorrect. This causes users to be overcharged

— RECOMMENDATION

Update the `positionIsClosed` flag after the GMX position is liquidated and the `curPositionKey` is cleared, or alternatively, deposit funds back into GMX immediately after liquidation so that future withdrawals trigger the expected callback and execution fee logic behaves correctly.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Deposit Reverts After Full Liquidation

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/PerpetualVaultperpetual-vault-rem-team1/blob/ba7b4bec49c42ed1ba440f5d00e8169ca39dabbb/contracts/PerpetualVault.sol#L766
REVIEW	Main Review

— DESCRIPTION

When a liquidation occurs, the vault processes it in `afterLiquidationExecution` and allows the keeper to create a new order using any remaining funds. However, if the vault is fully liquidated and no collateral is recovered, deposits become blocked.

This is due to a division-by-zero condition in the `_mint` function: `totalAmountBefore` becomes zero, while `totalShares` remains greater than zero.

As a result, the share calculation:

```
_shares = amount * totalShares / totalAmountBefore
```

Will revert due to a division by zero error, preventing any new deposits.

— RECOMMENDATION

Add a case to handle the scenario where `totalAmountBefore` is zero but `totalShares` is non-zero. Depending on the desired behavior the depositor can receive the entire vault share or a diluted amount of it.

— RESOLUTION

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

ADLs May Cause Excessive Fees

CATEGORY	Warning
LOCATION	PerpetualVault.sol
REVIEW	Main Review

— DESCRIPTION

When an ADL occurs during the last step of a withdrawal action the entire withdrawal flow is re-started to account for any tokens which might have been sent to the `PerpetualVault` as a result of the ADL.

This however can mean that a withdrawal action can expend even more than the `executionFee` allocated for the settle and decrease actions.

— RECOMMENDATION

Be aware of this behavior and be sure to top up the `GmxProxy` contract accordingly in the event that this occurs.

— RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

totalDepositAmount Not Decrementated In Insolvency

CATEGORY	Logical Error
LOCATION	PerpetualVault.sol
REVIEW	Main Review

— DESCRIPTION

In the `_handleReturn` function the `_transferToken` function is only invoked when the withdrawal amount is nonzero. If a withdrawal amount is zero e.g. due to an insolvent liquidation of the vault, then the `totalDepositAmount` decrement inside of the `_transferToken` function will not be reached.

— RECOMMENDATION

Consider refactoring the `_transferToken` function such that it handles the 0 amount case gracefully, decrementing the `totalDepositAmount` in all cases and early returning if the specified amount is 0.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.